

# Deep Learning and neural networks

Word representation by **word embeddings**

- Without manual feature engineering
- Allows generalizations ('ideal' and 'colorless' are adjectives, 'cat' and 'kitten' are felines, 'sunny' and 'sleet' are weather terms, 'awesome' is opposite to 'cringeworthy')
- Encode a word as input vector for neural network

# Word / Phrase as a vector

## ① One-hot vector

- ▶ i-th word in dictionary → bit 1 and bit 0 for remaining ones
- ▶ Problem: no similarity between words

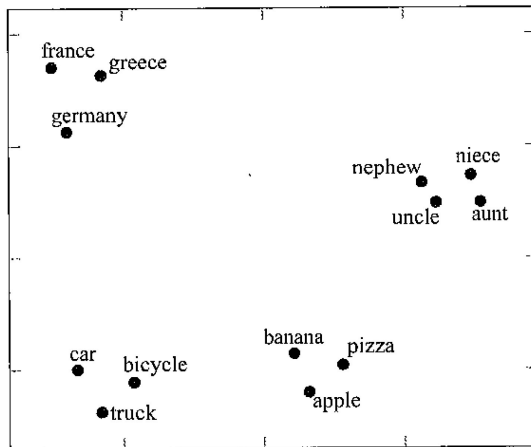
## ② N-gram counts (on words)

- ▶ Word in phrases
- ▶ For 100,000 words and 5-gram requires  $(10^5)^5 = 10^{25}$  grams (dimensions)

## ③ Word embedding

- ▶ A few hundred dimensions
- ▶ Automatically trained from data
- ▶ Vector of numbers, e.g. 'abacus' →  $[+0.7, -3.2, \dots]$
- ▶ No discernible meanings
- ▶ Similar words have similar vectors

# Word embeddings: example



# Word embeddings: properties

- Proximity of similar words
- Relationships
  - ▶ Let  $A$  stand for 'Athens' and  $B$  for 'Greece', then  $B - A$  stands for country/capital relationship
  - ▶ Same difference vector for France/Paris, Germany/Berlin, Zambia/Lusaka
  - ▶ Let  $C$  stand for 'Oslo' then  $B - A + C$  returns a vector near to 'Norway'
  - ▶ However: no guarantee for every relationship!
- Word embeddings helpful for all applications of deep learning to NLP tasks
- **Pretrained** vectors are often **generic** (WORD2VEC, GloVe, FASTTEXT)

# Word embeddings and difference vectors

| <b>A</b>    | <b>B</b>       | <b>C</b>  | <b><math>D = C + (B - A)</math></b> | <b>Relationship</b>     |
|-------------|----------------|-----------|-------------------------------------|-------------------------|
| Athens      | Greece         | Oslo      | Norway                              | <i>Capital</i>          |
| Astana      | Kazakhstan     | Harare    | Zimbabwe                            | <i>Capital</i>          |
| Angola      | kwanza         | Iran      | rial                                | <i>Currency</i>         |
| copper      | Cu             | gold      | Au                                  | <i>Atomic Symbol</i>    |
| Microsoft   | Windows        | Google    | Android                             | <i>Operating System</i> |
| New York    | New York Times | Baltimore | Baltimore Sun                       | <i>Newspaper</i>        |
| Berlusconi  | Silvio         | Obama     | Barack                              | <i>First name</i>       |
| Switzerland | Swiss          | Cambodia  | Cambodian                           | <i>Nationality</i>      |
| Einstein    | scientist      | Picasso   | painter                             | <i>Occupation</i>       |
| brother     | sister         | grandson  | granddaughter                       | <i>Family Relation</i>  |
| Chicago     | Illinois       | Stockton  | California                          | <i>State</i>            |
| possibly    | impossibly     | ethical   | unethical                           | <i>Negative</i>         |
| mouse       | mice           | dollar    | dollars                             | <i>Plural</i>           |
| easy        | easiest        | lucky     | luckiest                            | <i>Superlative</i>      |
| walking     | walked         | swimming  | swam                                | <i>Past tense</i>       |

# POS: Part of speech I

Task of POS-tagging: Predict correct part of speech for each word in a sentence

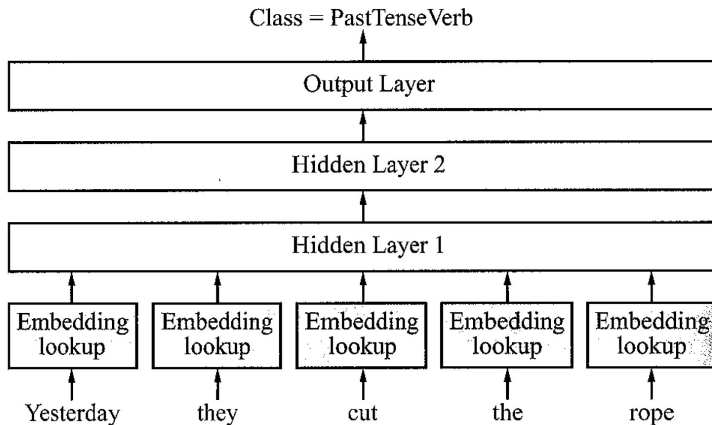
| Tag  | Word           | Description              |
|------|----------------|--------------------------|
| CC   | <i>and</i>     | Coordinating conjunction |
| CD   | <i>three</i>   | Cardinal number          |
| DT   | <i>the</i>     | Determiner               |
| EX   | <i>there</i>   | Existential there        |
| FW   | <i>per se</i>  | Foreign word             |
| IN   | <i>of</i>      | Preposition              |
| JJ   | <i>purple</i>  | Adjective                |
| JJR  | <i>better</i>  | Adjective, comparative   |
| JJS  | <i>best</i>    | Adjective, superlative   |
| LS   | <i>I</i>       | List item marker         |
| MD   | <i>should</i>  | Modal                    |
| NN   | <i>kitten</i>  | Noun, singular or mass   |
| NNS  | <i>kittens</i> | Noun, plural             |
| NNP  | <i>Ali</i>     | Proper noun, singular    |
| NNPS | <i>Fords</i>   | Proper noun, plural      |
| PDT  | <i>all</i>     | Predeterminer            |
| POS  | <i>'s</i>      | Possessive ending        |
| PRP  | <i>you</i>     | Personal pronoun         |
| \$   | <i>\$</i>      | Dollar sign              |
| "    | <i>'</i>       | Left quote               |
| (    | <i>[</i>       | Left parenthesis         |
| ,    | <i>,</i>       | Comma                    |
| :    | <i>;</i>       | Mid-sentence punctuation |

# POS: Part of speech II

POS-tagging is not trivial!

- One word can be tagged in different ways
- Example: cut → present-tense-verb, past-tense-verb, infinitive verb, past participle, adjective, noun
- Contextual words are essential

# POS: Part of speech III



# POS: Part of speech IV

## Steps:

- 1 Choose window width  $w$  (typically  $w = 5$  for 2 words left and right) and split every sentence of training corpus into overlapping windows with corresponding POS category of the middle word
- 2 Create vocabulary of all frequent words, total number is  $v$
- 3 Sort vocabulary in any arbitrary order
- 4 Choose value  $d$  as size of embedding vector

# POS: Part of speech V

- 5 Create  $v$ -by- $d$  weight matrix  $E$ : word embedding matrix initialized randomly or from pretrained vectors
- 6 Set up neural network (see slide)
  - ▶ First layer are  $w$  copies of  $E$
  - ▶ Two hidden layers  $z_1, z_2$  with weight matrices  $W_1, W_2$ , respectively
  - ▶ Output layer: softmax layer yielding probabilities over POS categories for middle word
- 7 Encode sequence of  $w$  words: concatenate corresponding embedding vectors
- 8 Train  $E, W_1, W_2, W_{out}$  using gradient descent

# Recurrent Neural Networks for NLP I

- Language consists of ordered sequences of words
- Context is extremely important
- For complex NLP task 5 words are not enough: dozens of words required
- Example “Eduardo told me that Miguel was very sick so I took **him** to the hospital” → “him” refers to “Miguel” and not to “Eduardo”

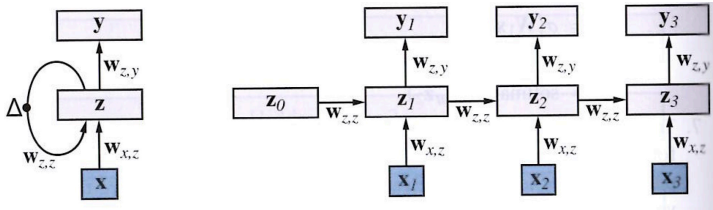
# Recurrent Neural Networks for NLP II

NLP task: create a language model

- Probability distribution over sequences of words
- Predicts next word given previous words
- Feedforward network:
  - ▶ **Fixed** window width is not appropriate: too small or too many parameters
  - ▶ Problem: weights for 'him' must be learnt for different word positions

# Recurrent Neural Networks for NLP III

One word at a time



- Hidden layer  $z$  has recurrent connection,  $\Delta$  means delay,  $x$  is input word embedding vector for next word in sentence
- Right figure: unrolled network for three timesteps (weights are shared)

# Recurrent Neural Networks for NLP IV

## Architecture

- Each word input is word embedding vector  $x$
- Hidden layer  $z$  as input for the next time step
- Target: multiclass classification where classes are words from vocabulary
- Output  $y$  is softmax probability distribution over next word in sentence
- RNN solves the problem of too many parameters: size of  $W_{x,z}$ ,  $W_{z,z}$ ,  $W_{z,y}$  is constant regardless of number of words
- In theory: regarding the whole history, but in practice restriction by limited memory for  $W_{x,z}$ ,  $W_{z,z}$ ,  $W_{z,y}$

# Recurrent Neural Networks for NLP V

## Training of the RNN

- Input words  $x_t$  of training corpus
- Output word  $x_{t+1}$
- Difference between required and actual output back-propagated through time (same weights for all time steps)

# Recurrent Neural Networks for NLP VI

Run the trained RNN

- Start with initial word  $x_1$
- $y_1$  is probability distribution over words
- Sample word from probability distribution and feed it back as  $x_2$  and so on

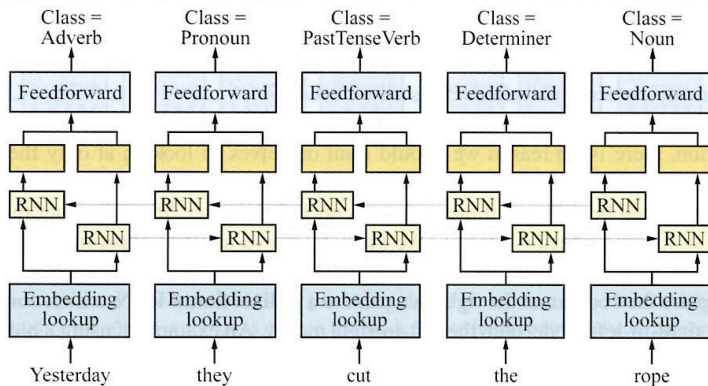
# Recurrent Neural Networks for Classification I

## Classification tasks

- Output is probability distribution over POS tags or coreference resolution
- Example for coreference resolution: Given 'Eduardo told me that Miguel was very sick so I took **him** to the hospital', input 'him' yields 'Miguel'
- Training is done analogously as with language model
- However, for POS it is essential to look backward and forward  
     $\leadsto$  bidirectional RNN

# Recurrent Neural Networks for Classification II

## Bidirectional RNN



# LSTMs for NLP task

- Problem with RNN: solves **limited** context problem
- Information gets distorted or lost after a long number of steps (see game of telephone)
- Problem is similar to problem of vanishing gradients of many layers
- Solution: **Long short-term memory** (LSTM) model
  - ▶ RNN with gating units
  - ▶ LSTM can remember, copy and forget parts of input
- Example: 'The athletes, who all won their local qualifiers and advanced to the finals in Tokyo, now ...'  
what is more probable, 'compete' or 'competes'?

# Sequence-to-Sequence Models I

NLP: natural language processing

- NLP task **machine translation** (MT): translate sentence from source language to target language
- Train MT with large corpus of source/target sentence pairs
- Goal: translate new sentence that is not in training data

# Sequence-to-Sequence Models II

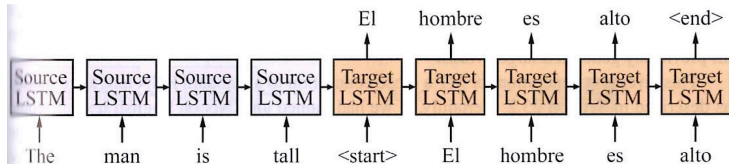
## RNN for MT task

- If correspondence between source words and target words were one-to-one correspondence  
     $\leadsto$  simple tagging task
- However: **no one-to-one** correspondence, see
  - ▶ Many-to-one: Spanish 'caballo de mar'  $\leftrightarrow$  'seahorse'
  - ▶ Reversed order: Spanish 'perro grande'  $\leftrightarrow$  'big dog'
  - ▶ Extreme word reordering: in English subject at start and in Fijian at the end
- Therefore:
  - ▶ Generate one word at time but remember source parts not translated yet and remember parts already translated
  - ▶ Sometimes process entire source sentence before generate target
  - ▶ General: **Each target word is conditional on entire source sentence and on all generated target words**  
     $\leadsto$  use RNN

# Sequence-to-Sequence Models III

Simple translation from source language to target language

- RNN for source language as input for another RNN for target language  $\leadsto$  **sequence-to-sequence model**



# Sequence-to-Sequence Models IV

## Drawbacks of sequence-to-sequence models

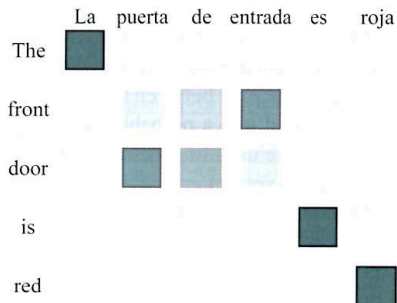
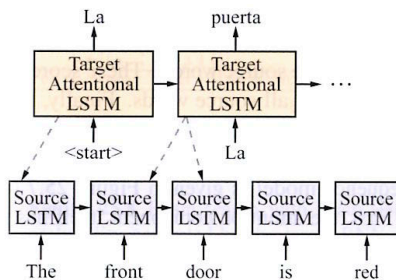
- **Nearby context bias:** nearby context is stronger than far-away context but far-away context can be crucial as well
- **Fixed context limit:** entire source sentence stored in single fixed-dimensional hidden state vector ( $\approx 1024$  dimensions)  
 $\leadsto$  increase vector size  $\rightarrow$  slow training and overfitting
- **Slow sequential processing:** processing one word at time (no batch processing)

# Attentional sequence-to-sequence model I

Idea: hidden vectors of source RNN as input for target RNN

- Mitigates shortcomings of sequence-to-sequence models (which uses only previous vector)
- Concatenate all source RNN hidden vectors
  - ~> huge increase in number of weights
  - ~> high computation costs and risk of overfitting
- However: only small part of source is relevant to each target word
- Different source parts for every target word
- Part ~> **attention**

# Attentional sequence-to-sequence model II



# Attentional sequence-to-sequence model III

Formalization:

- Standard target RNN:  
 $\mathbf{h}_i = RNN(\mathbf{h}_{i-1}, \mathbf{x}_i)$  where  $\mathbf{h}_i$  is target word vector at time stamp  $i$   
and  $\mathbf{x}_i$  is input word vector (bold font for vector!)
- Target RNN for attentional sequence-to-sequence model:  
 $\mathbf{h}_i = RNN(\mathbf{h}_{i-1}, [\mathbf{x}_i; \mathbf{c}_i])$  where  $\mathbf{c}_i$  is context vector and  $[\cdot; \cdot]$  is concatenation

# Attentional sequence-to-sequence model IV

Definition of context vector  $\mathbf{c}_i$

- raw attention score (attention matrix):  $r_{ij} = \mathbf{h}_{i-1} \cdot \mathbf{s}_j$   
scalar product with source RNN word vector  $\mathbf{s}_j$   
(remember word embeddings!)
- Normalizing by softmax:  $a_{ij} = e^{r_{ij}} / (\sum_k e^{r_{ik}})$   
 $\leadsto$  probabilities
- Weighted average of source RNN vectors as context vector:  
 $\mathbf{c}_i = \sum_j a_{ij} \cdot \mathbf{s}_j$

# Decoding I

Decoding means generation of target sentence word by word from probability distribution

- Simplest form: select word with highest probability  
     $\leadsto$  greedy decoding
- Goal of decoding: maximize probability of entire target sentence
- Problem: greedy decoding may not achieve goal
  - ▶ 'The front'  $\rightarrow$  'La entrada'
  - ▶ 'The front door'  $\rightarrow$  'La puerta de entrada'
- Attention cannot solve the problem because sometimes you must see the end before you can start the target sentence

# Decoding II

Better approach: **beam search**

- Keep top  $k$  hypotheses at each stage
- Extend each by one word and choose the best  $k$  from the  $k^2$  new hypotheses and so on
- At the end: take highest scoring hypothesis

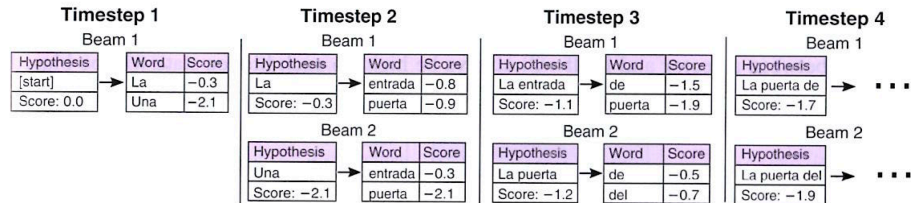
# Decoding III

Example translation:

'The front door is open'  $\rightarrow$  'La puerta de entrada es roja'

but 'front'  $\rightarrow$  'entrada'

$k=2$



# The Transformer Architecture I

see paper "Attention is all you need" (Vaswani et al., 2018) and youtube video (Hedu AI)

## Self-attention

- Attention on words from source sequence (encoder)
- Attention captures long-distance and nearby context
- Source words against source words
- Result is attention filter
- Input: source word vectors in positional embedding (capturing word order)
- Three linear mappings (matrices) on input:
  - ▶ Query: projecting one word ( $d$  dimensions):  $\mathbf{q}_i = \mathbf{W}_q \mathbf{x}_i$
  - ▶ Key: against projected other word ( $d$  dimensions):  $\mathbf{k}_i = \mathbf{W}_k \mathbf{x}_i$
  - ▶ Value: further representation of word ( $d$  dimensions):  $\mathbf{v}_i = \mathbf{W}_v \mathbf{x}_i$

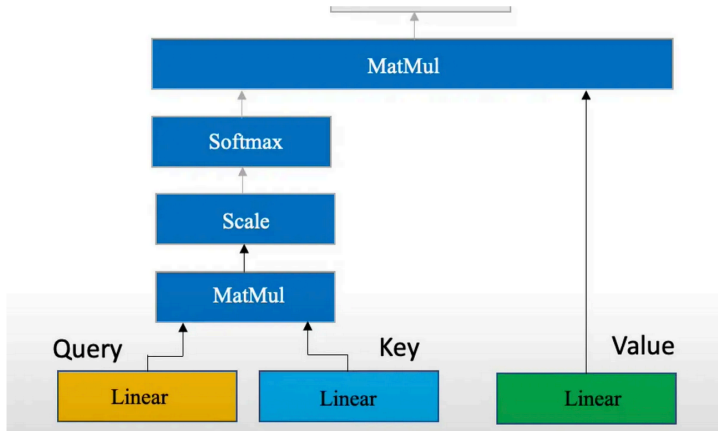
# The Transformer Architecture II

## Attention filter

- Similarity between projected word vectors:  
$$r_{ij} = (\mathbf{q}_i \cdot \mathbf{k}_j) / \sqrt{d}$$
- Asymmetry:  $r_{ij} \neq r_{ji}$
- Softmax scaling provides attention filter:  $a_{ij} = e^{r_{ij}} / (\sum_k e^{r_{ik}})$
- Context as attention:  $\mathbf{c}_i = \sum_j a_{ij} \cdot \mathbf{v}_j$

# The Transformer Architecture III

Attention filter Query/Key on Value



# The Transformer Architecture IV

Encoder:

Multiheaded attention

- Many attentions for capturing of sentence semantics
- Several attention filters (for example 8) on copied inputs

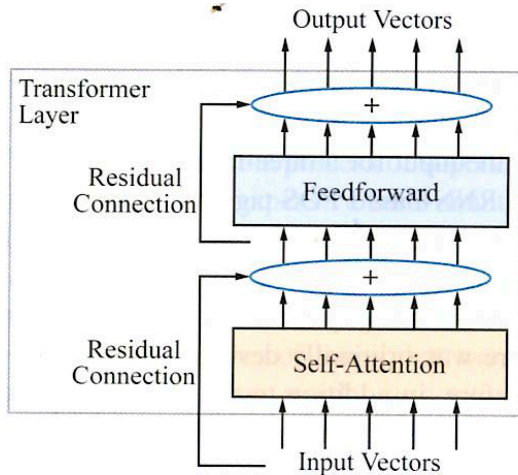
Residual connection

- Problem of vanishing gradients and loss of information over many network layers
- Solution: Residual connection: direct path between far away layers

Feedforward with several nn layers and ReLU

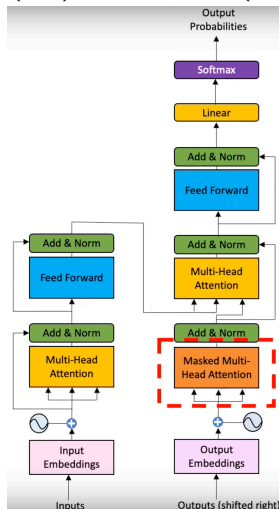
# The Transformer Architecture V

## Encoder



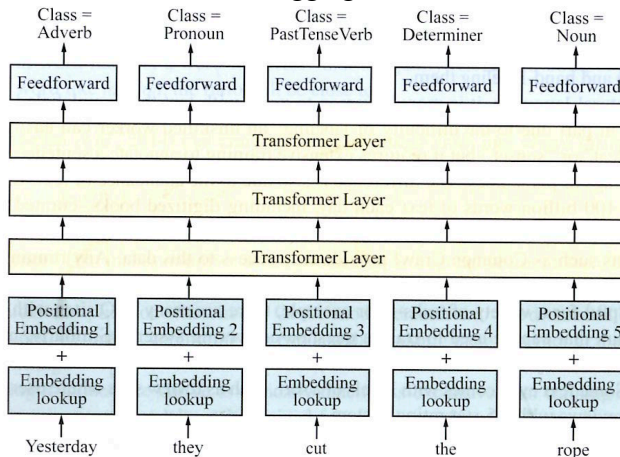
# The Transformer Architecture VI

Transformer with encoder (left) and decoder (right) for translation



# The Transformer Architecture VII

## Transformer architecture for POS tagging



# Problems of LLMs

## Some problems

- Huge number of neurons needs to be trained
- Training needs
  - ▶ huge document collection
  - ▶ huge computation power
  - ▶ Controlled semantic of document collection (de-biasing, integrity, compliance, ...)
- No real world understanding
  - ~ hallucinating, probabilistic parrot
  - ~ not reliable
- LLMs lack interpretability/explainability (black-box)

# LLM and IR

## Potential benefits of LLMs for Information Retrieval

→ LLM-driven IR

- Training of **word embedding** for vector space model
- Query **expansion** and **reformulation**
- **Interactions** with humans (for query and retrieval result)
  - ▶ Multimodal
  - ▶ Multilingual
  - ▶ Personalization
- **Evaluation** of IR systems
- **Ranking** of result documents
- **RAG**

# LLM and RAG I

Application of LLM in specific application domain, for example a chatbot

- 1st Problem: LLM often **hallucinates**  
    ~> big problem for question answering (QA)
- 2nd Problem: how to **tailor** it to domain specific knowledge  
    ~> **Fine tuning** an LLM is often unfeasible due to resource restrictions

Solution: **RAG** (Retrieval Augmented Generation) and LLM

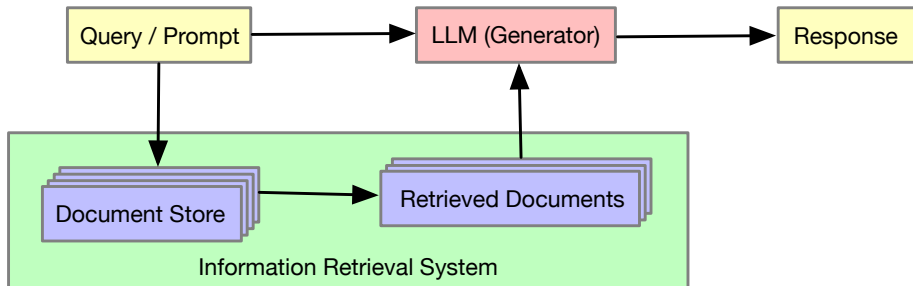
~> **RA-LLM**

# LLM and RAG II

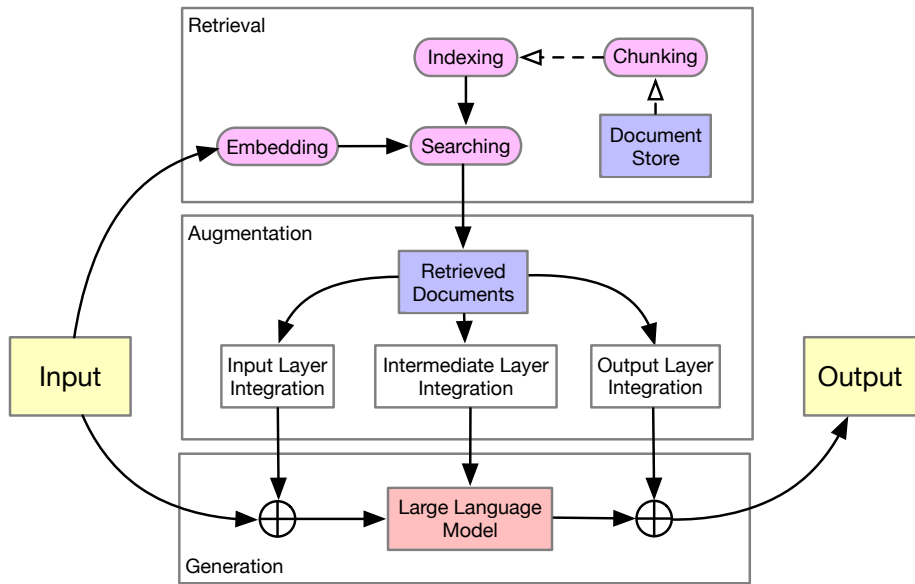
## Main concepts

- Combine LLM with **information retrieval system**
- LLM query/prompt is sent to IR system
- retrieved **relevant documents** are sent to the LLM
- LLM uses **context** (original query/prompt and relevant documents) for computing response
  
- Solution of 1st problem: LLM uses **knowledge** from query specific documents
- Solution of 2nd problem: IR system uses **contextual documents** (without the need of retraining)

# LLM and RAG III



# Retrieval, Augmentation and Generation I



# Retrieval, Augmentation and Generation II

## Retrieval granularity

- Documents are split into **chunks** (text fragments) of size less than a threshold
- Size has effect on **efficiency** and **effectiveness** of RA-LLM
- Chunks are **overlapping retrieval units**

# Retrieval, Augmentation and Generation III

## Two types of retrieval

- **Sparse retrieval:** **classical IR models** like TF-IDF, BM25
  - ▶ No training
  - ▶ Fixed evaluation of relevance
- **Dense retrieval:** **learnt embedding** into vector space
  - ▶ Trainable  $\leadsto$  more flexible
  - ▶ Embedding layer of LLM can be used

# Retrieval, Augmentation and Generation IV

**Augmentation** integrates retrieval and generation, three main designs:

- **Input-Layer Integration** for black-box LLMs (intermediate and output layer are not accessible)
  - ▶ **Combination** of retrieved chunks and query  
     $\leadsto$  **context**
  - ▶ Concatenated input can be **too long**
- **Output-Layer Integration**
  - ▶ **Joins** retrieval and generation result: **linear integration**
  - ▶ LLM remains unchanged
  - ▶ **No reasoning** about the text  
     $\leadsto$  post-hoc answer refinement mechanism performs reasoning
- **Intermediate-Layer Integration**
  - ▶ Additional complexity needs special **model training**
  - ▶ Requires **high access** to generation model

# Retrieval, Augmentation and Generation V

## Generation by an LLM

- **Training-free** RAG-method
  - ~> Input-layer integration
    - ▶ **Prompt engineering-based** method
    - ▶ **Retrieval-guided token generation** method
- **Training-based** RAG-method: training of IR system and LLM
  - ▶ **Independent** training
  - ▶ **Sequential** training
  - ▶ **Joint** training